

Vivado

Design Suite Mastery Course

Installation · CLI Simulation · GUI · Synthesis · Implementation · Bitstream · Debugging · IP Catalog

9 Chapters · xvlog · xelab · xsim · TCL Scripts · Full FPGA Workflow

© 2026 squared-studio · squared-studio.cc/SkillVerse

Introduction to Vivado

Vivado is a comprehensive design suite developed by Xilinx for the synthesis and analysis of hardware description language (HDL) designs. It is primarily used for FPGA (Field-Programmable Gate Array) and SoC (System on Chip) design and development. Vivado provides a robust environment for designing, simulating, and implementing digital circuits.

Features of Vivado

- **Integrated Design Environment (IDE):** Vivado offers a user-friendly IDE that integrates various design tools, making it easier to manage complex projects.
- **High-Level Synthesis (HLS):** Allows designers to convert C, C++, and SystemC code into HDL, enabling higher productivity and faster design cycles.
- **IP Integrator:** Facilitates the integration of pre-built IP (Intellectual Property) cores into designs, reducing development time.
- **Advanced Timing Analysis:** Provides detailed timing analysis to ensure that designs meet performance requirements.
- **Simulation and Debugging Tools:** Includes powerful simulation and debugging tools to verify and troubleshoot designs.
- **Support for Multiple Languages:** Supports VHDL, Verilog, and SystemVerilog, catering to a wide range of design preferences.
- **Scalability:** Capable of handling designs ranging from small FPGAs to large SoCs.

Use Cases of Vivado

- **FPGA Design:** Vivado is extensively used for designing and implementing FPGA-based systems, which are used in various applications such as telecommunications, automotive, and consumer electronics.
- **SoC Development:** The suite supports the development of complex SoCs, integrating multiple processors, peripherals, and custom logic.
- **Prototyping:** Vivado enables rapid prototyping of digital systems, allowing designers to test and validate their designs before final production.
- **High-Performance Computing:** Used in the development of high-performance computing systems, leveraging the parallel processing capabilities of FPGAs.
- **Embedded Systems:** Facilitates the design of embedded systems, combining hardware and software components for specialized applications.

Vivado's comprehensive toolset and advanced features make it an essential tool for modern digital design and development, enabling engineers to create efficient and high-performance systems.

Installing Vivado

Installing Vivado involves several steps to ensure that the software is correctly set up on your system. Follow the instructions below to install Vivado on your computer.

System Requirements

Before installing Vivado, ensure that your system meets the following requirements:

- **Operating System:** Windows 10/8.1/7, Linux (Red Hat Enterprise, CentOS, Ubuntu)
- **Memory:** Minimum 8 GB RAM (16 GB or more recommended)
- **Disk Space:** At least 50 GB of free disk space
- **Processor:** Intel or AMD x86 processor with support for SSE2
- **Additional Software:** Java Runtime Environment (JRE) 1.8 or later

Downloading Vivado

- Visit the Xilinx website.
- Navigate to the Vivado Design Suite section.
- Select the appropriate version of Vivado for your operating system.
- Download the installer package.

Installing Vivado on Windows

- Locate the downloaded installer package and double-click to run it.
- Follow the on-screen instructions to begin the installation process.
- Choose the installation directory and ensure you have sufficient disk space.
- Select the components you wish to install (e.g., Vivado, SDK, Documentation).
- Accept the license agreements and proceed with the installation.
- Once the installation is complete, launch Vivado from the Start menu.

Installing Vivado on Linux

- Open a terminal and navigate to the directory where the installer package is located.
- Make the installer executable by running the command:

```
chmod +x Xilinx_Vivado_<version>_Lin64.bin
```

- Run the installer with root privileges:
 - `sudo ./Xilinx_Vivado_<version>_Lin64.bin`
- Follow the on-screen instructions to complete the installation.
- After installation, you can launch Vivado by running:

```
source /opt/Xilinx/Vivado/<version>/settings64.sh
```

```
vivado &
```

Post-Installation Steps

- **Licensing:** Ensure that you have a valid license for Vivado. You can obtain a license from the Xilinx website and configure it using the Vivado License Manager.
- **Environment Setup:** Set up the environment variables required for Vivado to function correctly. This is typically done automatically during installation, but you can verify by checking the settings in your system's environment variables.

By following these steps, you will have Vivado installed and ready to use on your system. You can now proceed to create and manage your FPGA and SoC designs using the powerful tools provided by Vivado.

Command-Line Simulation

In this chapter, we will explore how to perform RTL simulation using Vivado's command-line tools: `xvlog`, `xelab`, and `xsim`. These tools provide a powerful way to automate and script your simulation workflows.

Overview of Command-Line Tools

- **xvlog**: Compiles Verilog and SystemVerilog source files.
- **xelab**: Elaborates the design and generates a simulation snapshot.
- **xsim**: Runs the simulation using the generated snapshot.

Setting Up the Environment

Before running the command-line tools, ensure that the Vivado environment is set up correctly. This can be done by sourcing the Vivado settings script:

```
source /opt/Xilinx/Vivado/<version>/settings64.sh
```

Compiling the RTL Sources

Use `xvlog` to compile your Verilog or SystemVerilog source files. For example:

```
xvlog -sv my_design.sv my_testbench.sv
```

This command compiles `my_design.sv` and `my_testbench.sv` files.

Elaborating the Design

After compiling the sources, use `xelab` to elaborate the design and create a simulation snapshot. For example:

```
xelab -debug typical my_testbench -s my_sim_snapshot
```

This command elaborates the `my_testbench` module and creates a simulation snapshot named `my_sim_snapshot`.

Running the Simulation

Finally, use `xsim` to run the simulation with the generated snapshot. For example:

```
xsim my_sim_snapshot -gui
```

This command runs the simulation and opens the GUI for waveform analysis.

Common Options for Command-Line Tools

Here are some common options for `xvlog`, `xelab`, and `xsim` that you might find useful:

xvlog Options

- **-sv**: Specifies that the input files are SystemVerilog files.
- **-d** : Defines a macro for conditional compilation.
- **-i** : Specifies the directory to search for include files.

Example:

```
xvlog -sv -d SIMULATION -i ./include my_design.sv
```

xelab Options

- **-debug** : Specifies the debug level (e.g., typical, full) for the simulation.
- **-s** : Names the simulation snapshot.

Example:

```
xelab -debug typical my_testbench -s my_sim_snapshot
```

xsim Options

- **-gui**: Launches the simulation in the GUI mode.
- **-R**: Runs the simulation without entering the interactive mode.

Example:

```
xsim my_sim_snapshot -R -gui
```

By using these options, you can customize and control the behavior of the Vivado command-line tools to better suit your simulation needs.

Automating the Workflow with a Script

You can automate the entire simulation workflow by creating a shell script. Here is an example script:

```
#!/bin/bash
```

```
# Source the Vivado settings
```

```
source /opt/Xilinx/Vivado/<version>/settings64.sh
```

```
# Compile the RTL sources
```

```
xvlog -sv my_design.sv my_testbench.sv
```

Elaborate the design

```
xelab -debug typical my_testbench -s my_sim_snapshot
```

Run the simulation

```
xsim my_sim_snapshot -gui
```

Save this script as run_simulation.sh, make it executable, and run it:

```
chmod +x run_simulation.sh
```

./run_simulation.sh

By following these steps, you can perform RTL simulation using Vivado's command-line tools, allowing for greater flexibility and automation in your design verification process.

Simulating a Simple RTL on Vivado GUI

In this chapter, we will walk through the steps to simulate a simple RTL (Register Transfer Level) design using the Vivado GUI. Simulation is a crucial step in verifying the functionality of your design before implementing it on hardware.

Creating a New Project

- Launch Vivado and select "Create New Project" from the Quick Start menu.
- Enter a name for your project and choose a location to save it.
- Select "RTL Project" and check the box for "Do not specify sources at this time."
- Click "Next" and choose the appropriate part for your FPGA board.
- Click "Finish" to create the project.

Adding RTL Sources

- In the Flow Navigator, click on "Add Sources."
- Select "Add or Create Design Sources" and click "Next."
- Click "Add Files" and browse to the location of your RTL files (e.g., Verilog or VHDL files).
- Select the files and click "OK," then click "Finish" to add the sources to your project.

Adding Simulation Sources

- In the Flow Navigator, click on "Add Sources."
- Select "Add or Create Simulation Sources" and click "Next."
- Click "Add Files" and browse to the location of your testbench files.
- Select the files and click "OK," then click "Finish" to add the simulation sources to your project.

Running the Simulation

- In the Flow Navigator, click on "Run Simulation" under the "Simulation" section.
- Select "Run Behavioral Simulation" to start the simulation process.
- The Vivado simulator will launch, and you will see the simulation waveform window.
- Use the waveform window to analyze the signals and verify the functionality of your design.

Analyzing Simulation Results

- Use the zoom and pan tools in the waveform window to navigate through the simulation results.
- Check the signal values at different time points to ensure that your design behaves as expected.
- If you find any issues, go back to your RTL code, make the necessary changes, and re-run the simulation.

By following these steps, you can simulate and verify your RTL design using the Vivado GUI. Simulation helps you catch and fix issues early in the design process, ensuring a smoother implementation on hardware.

Synthesis in Vivado

In this chapter, we will cover the steps to synthesize an RTL design using the Vivado GUI. Synthesis is the process of converting your RTL code into a gate-level netlist that can be implemented on an FPGA.

Setting Up the Synthesis

- Open your project in Vivado.
- In the Flow Navigator, click on "Run Synthesis" under the "Synthesis" section.
- Vivado will start the synthesis process, converting your RTL code into a gate-level netlist.

Analyzing Synthesis Results

- Once the synthesis is complete, Vivado will display the Synthesis Completed dialog.
- Click "Open Synthesized Design" to view the synthesized netlist.
- Use the various analysis tools available in Vivado to inspect the synthesized design, such as the Schematic Viewer and the Timing Summary.

Synthesis Settings

- In the Flow Navigator, click on "Settings" under the "Project Manager" section.
- Navigate to the "Synthesis" settings to configure various synthesis options, such as optimization goals and constraints.
- Adjust the settings as needed to meet your design requirements.

Common Synthesis Options

Here are some common options you might use during synthesis:

- **-flatten_hierarchy**: Controls the flattening of the design hierarchy.
- **-opt_mode**: Specifies the optimization mode (e.g., Area, Performance).
- **-retiming**: Enables or disables retiming optimization.

Example:

```
synth_design -top top_module -flatten_hierarchy full -opt_mode Performance -retiming
```

By following these steps, you can synthesize your RTL design using Vivado and analyze the results to ensure your design meets the required specifications.

Implementation in Vivado

In this chapter, we will cover the steps to implement an RTL design using the Vivado GUI. Implementation involves placing and routing the synthesized netlist onto the FPGA fabric.

Setting Up the Implementation

- Open your project in Vivado.
- In the Flow Navigator, click on "Run Implementation" under the "Implementation" section.
- Vivado will start the implementation process, which includes placement and routing of the design.

Analyzing Implementation Results

- Once the implementation is complete, Vivado will display the Implementation Completed dialog.
- Click "Open Implemented Design" to view the implemented design.
- Use the various analysis tools available in Vivado to inspect the implemented design, such as the Device Viewer and the Timing Summary.

Implementation Settings

- In the Flow Navigator, click on "Settings" under the "Project Manager" section.
- Navigate to the "Implementation" settings to configure various implementation options, such as placement and routing strategies.
- Adjust the settings as needed to meet your design requirements.

Common Implementation Options

Here are some common options you might use during implementation:

- **-directive**: Specifies the implementation directive (e.g., Performance_Explore, Area_Explore).
- **-place_design**: Controls the placement of the design.
- **-route_design**: Controls the routing of the design.

Example:

```
place_design -directive Performance_Explore  
route_design -directive Performance_Explore
```

By following these steps, you can implement your RTL design using Vivado and analyze the results to ensure your design meets the required specifications.

Bitstream Generation in Vivado

In this chapter, we will cover the steps to generate a bitstream file from your implemented design using the Vivado GUI. The bitstream file is used to program the FPGA with your design.

Generating the Bitstream

- Open your project in Vivado.
- In the Flow Navigator, click on "Generate Bitstream" under the "Program and Debug" section.
- Vivado will start the bitstream generation process, which includes writing the configuration data to a file.

Analyzing Bitstream Generation Results

- Once the bitstream generation is complete, Vivado will display the Bitstream Generation Completed dialog.
- Click "Open Implemented Design" to view the implemented design if you haven't already.
- Use the various analysis tools available in Vivado to inspect the final design and ensure it meets your requirements.

Programming the FPGA

- Connect your FPGA board to your computer.
- In the Flow Navigator, click on "Open Hardware Manager" under the "Program and Debug" section.
- Click "Open Target" and select "Auto Connect" to connect to your FPGA board.
- Click "Program Device" and select the generated bitstream file.
- Click "Program" to configure the FPGA with your design.

By following these steps, you can generate a bitstream file from your implemented design and program your FPGA using Vivado.

Debugging in Vivado

In this chapter, we will cover the steps to debug your design using the Vivado GUI. Debugging helps you identify and fix issues in your design.

Setting Up Debug Probes

- Open your project in Vivado.
- In the Flow Navigator, click on "Set Up Debug" under the "Program and Debug" section.
- Select the signals you want to monitor and add them as debug probes.
- Click "OK" to insert the debug cores into your design.

Running the Debug

- In the Flow Navigator, click on "Run Implementation" to re-implement the design with the debug probes.
- Once implementation is complete, generate the bitstream and program the FPGA as described in the previous chapter.
- In the Flow Navigator, click on "Open Hardware Manager" and connect to your FPGA board.
- Click "Run Trigger" to start capturing data from the debug probes.

Analyzing Debug Results

- Use the waveform viewer in the Hardware Manager to analyze the captured data.
- Check the signal values and timing to identify any issues in your design.
- If you find any issues, go back to your RTL code, make the necessary changes, and re-run the implementation and debugging process.

By following these steps, you can debug your design using Vivado and ensure it functions correctly on the FPGA.

IP Catalog and Integration in Vivado

In this chapter, we will explore how to use the IP Catalog in Vivado to integrate pre-built Intellectual Property (IP) cores into your design. The IP Catalog provides a wide range of IP cores that can be easily configured and added to your project, saving development time and effort.

Accessing the IP Catalog

- Open your project in Vivado.
- In the Flow Navigator, click on "IP Catalog" under the "Project Manager" section.
- The IP Catalog window will open, displaying a list of available IP cores organized by category.

Adding an IP Core to Your Design

- In the IP Catalog, browse or search for the IP core you want to add to your design.
- Double-click on the IP core to open the "Customize IP" dialog.
- Configure the IP core parameters as needed for your design requirements.
- Click "OK" to add the IP core to your project.

Integrating the IP Core

- In the Flow Navigator, click on "Add Sources" and select "Add or Create Design Sources."
- Click "Next" and then "Add Files" to add the IP core files to your project.
- Click "Finish" to complete the integration process.
- Use the IP Integrator tool to connect the IP core to other parts of your design.

Generating the IP Core

- In the Flow Navigator, click on "Generate Output Products" under the "IP Integrator" section.
- Select the IP core and click "Generate" to create the necessary files for the IP core.
- Once the generation process is complete, the IP core will be ready for use in your design.

Example: Adding a FIFO IP Core

- Open the IP Catalog and search for "FIFO."
- Double-click on the "FIFO Generator" IP core to open the "Customize IP" dialog.
- Configure the FIFO parameters, such as data width and depth, according to your design requirements.
- Click "OK" to add the FIFO IP core to your project.
- Use the IP Integrator tool to connect the FIFO IP core to other modules in your design.

By following these steps, you can efficiently integrate IP cores from the Vivado IP Catalog into your design, leveraging pre-built components to accelerate your development process.